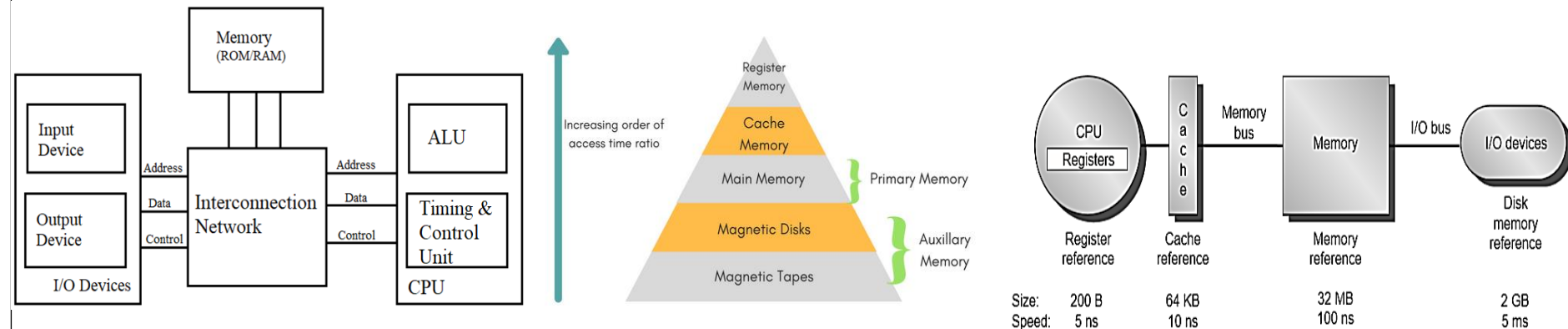


Memory Hierarchy

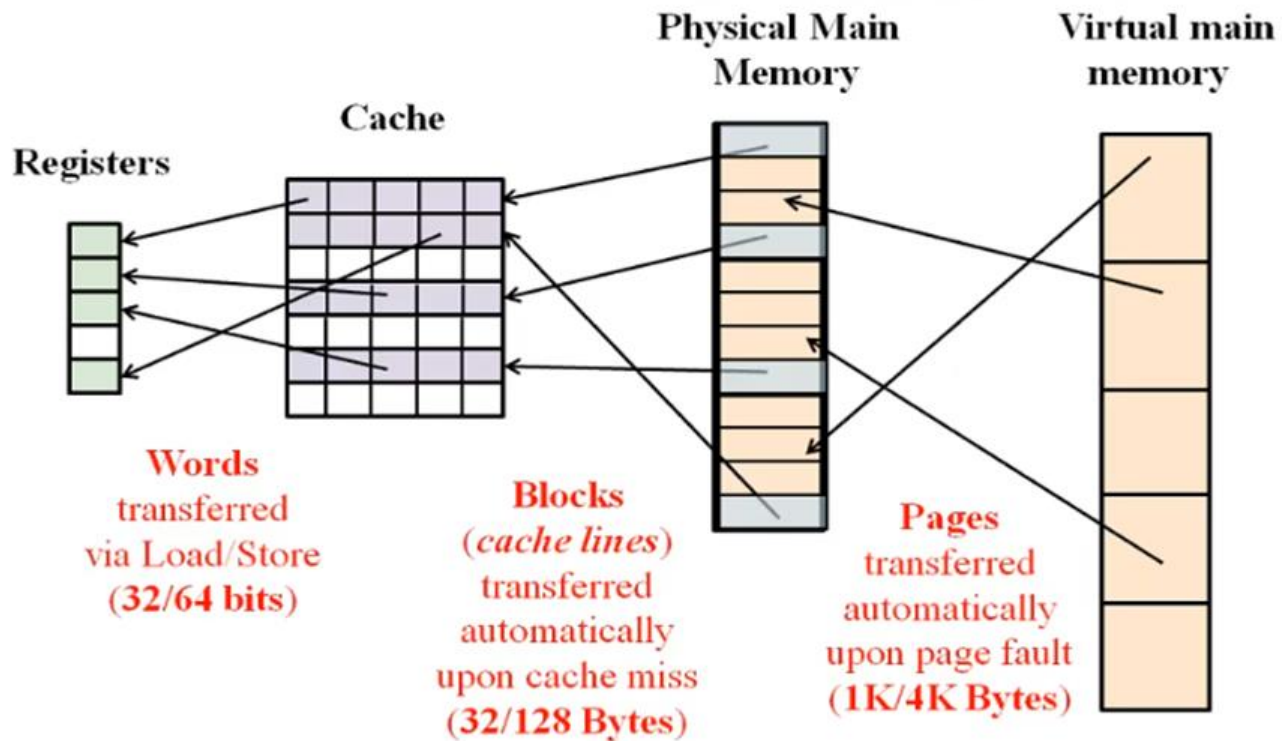
Memory hierarchy

- Memory is very important part of Computer
- Faster memories are available in smaller number of bits per chip



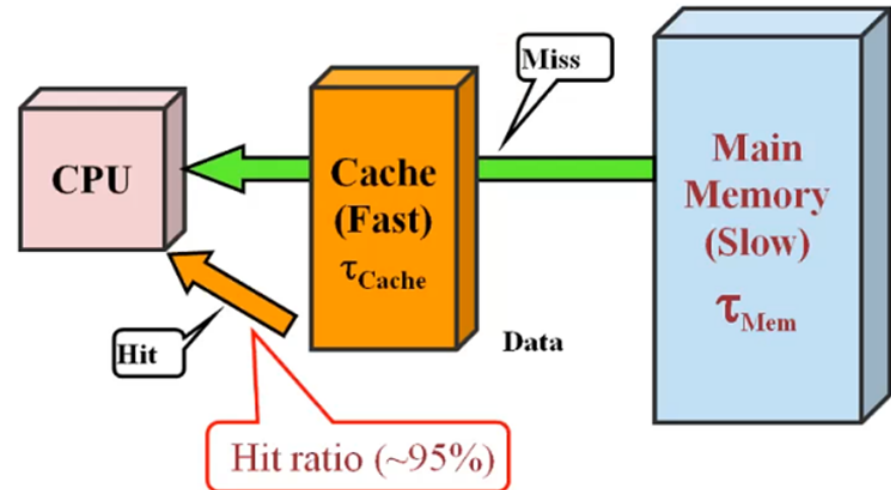
- **Principle of locality:** Data most recently used is very likely to be accessed again in near future
- Keep recently-used data in fastest memory (smaller memory close to CPU)
- Keep data not-used-recently in slower memory (larger memory farther away from CPU)

Memory Hierarchy



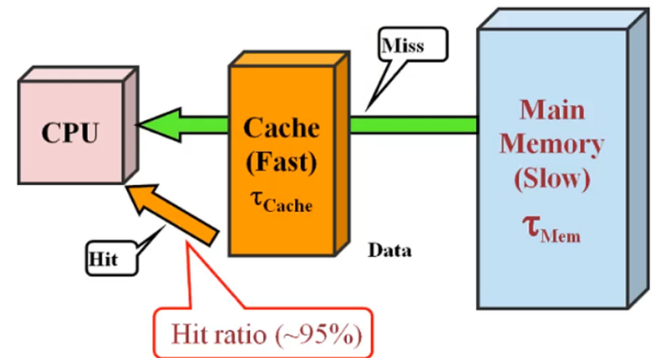
Working of Cache Memory

- Cache is first level of memory hierarchy
- Cache memory: Small, fast memory close to CPU (holding most recently used data/code)
- **Cache hit:** If CPU finds requested data (referenced by a program) in cache memory
- **Cache miss:** If CPU doesn't find requested data in cache memory
- When cache miss happens, block of data (called block) containing the requested data is retrieved from main memory and placed in cache

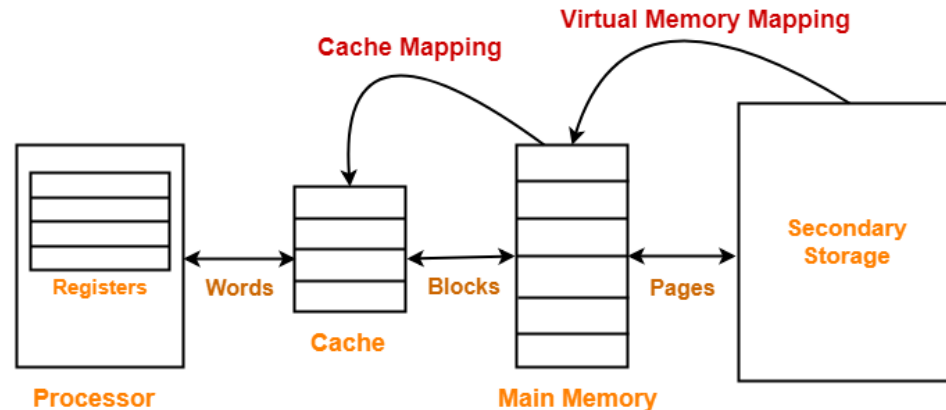


Cache hit, cache miss, page fault

- Temporal locality: retrieved data is likely to be used again in near future
- Spatial locality: high probability that other data within the block will be used soon
- Cache miss handled by hardware. CPU is stalled until requested data available
- **Page fault:** If CPU doesn't find requested data in cache and main memory
- Virtual address space broken into multiple pages
- When page fault happens, page containing the requested data is retrieved from disc memory and placed in main memory
- Page fault handled by software. CPU is not stalled but switched to other task until requested data available



Properties of Cache Memory



The **size** of an element in a cache is called a **block** or **line**

Line or Block size is given in terms of words

- E.g: Word = 32 bytes for a 32 bit processor

Whenever a location in **MM** is to be accessed

- The entire block/line of the cache is filled from the MM
- Even if the processor wants to read a byte in that line, whole line is brought to Cache from MM

- Block size (size of a cache line/block)
- Cache Size (total cache size)
- Mapping functions (**MM** address to Cache)

Cache mapping techniques with Main Memory (MM)

Cache is volatile and empty when power is switched on .

Cache write operation : Where to place a block in cache

Based upon locality of reference instructions and data are picked up and placed in cache depending upon three main placement/mapping techniques.

Direct Mapped: When a block from MM is always mapped on a one place in the cache

Fully Associative: When any block of MM is mapped to any place (block) in cache

Set Associative: In this the combined effect of direct mapping and fully associative mapping is to be used

Where can a block be placed in a cache?

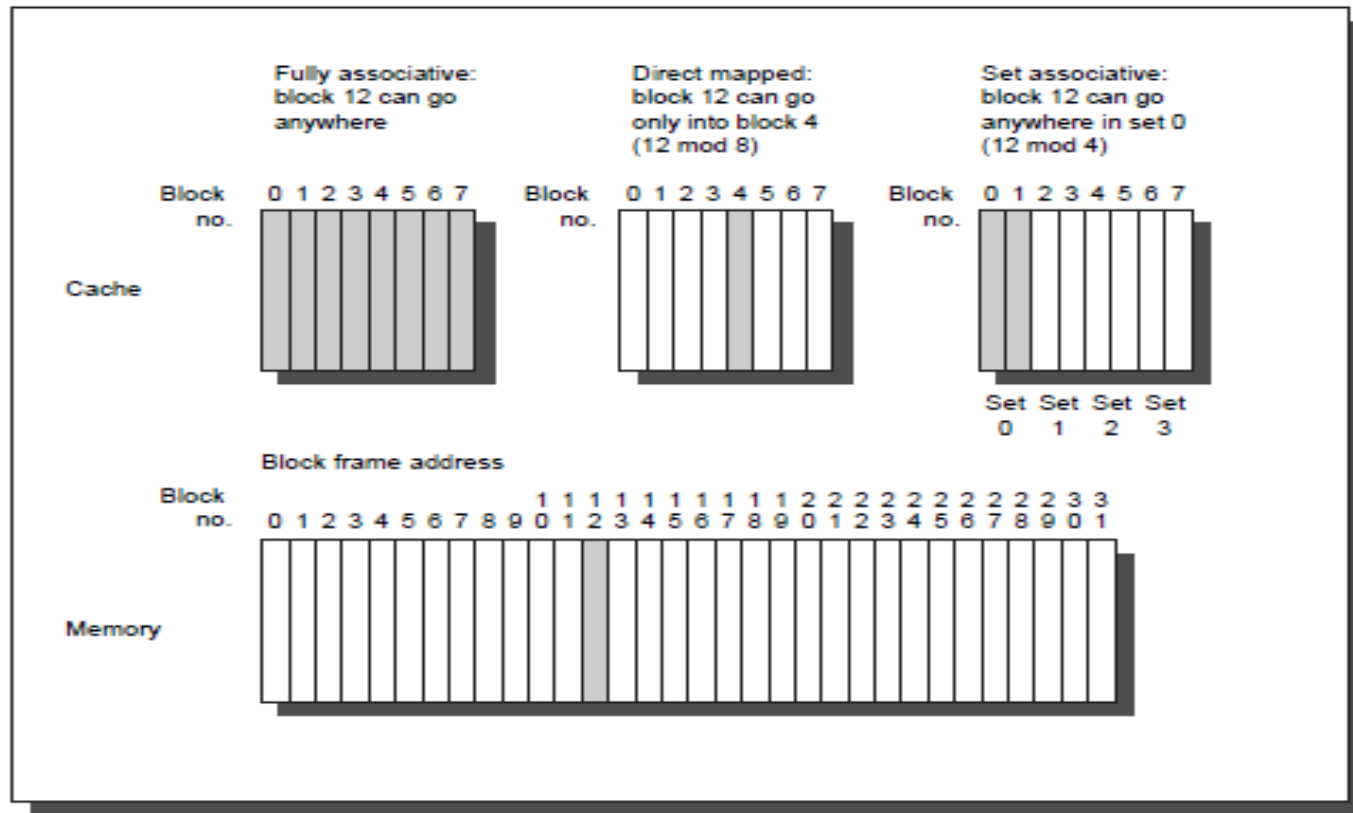
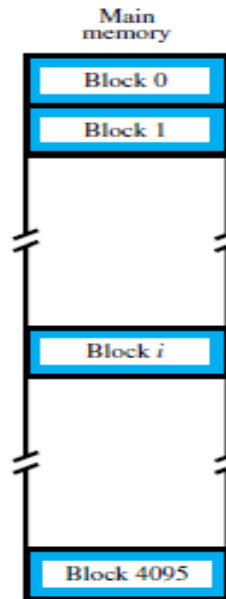
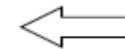
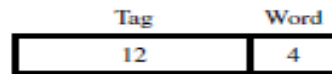
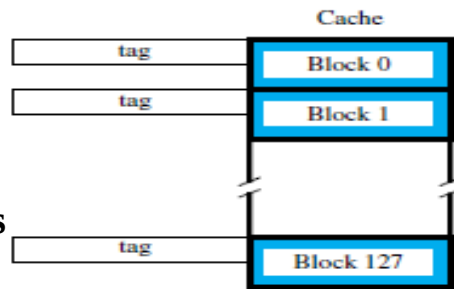


FIGURE 5.2 This example cache has eight block frames and memory has 32 blocks.

Associative or Fully Associative Mapping

- 2048 or 2K words
- 128 blocks of 16 words each



- 64K words
(4K blocks of 16 words each)

Main memory address

Associative or Fully Associative Mapping

Consider Pentium Processor Cache

Size of Main Mem:	4GB = 2^{32}
Size of Cache Mem:	8KB = 2^{13}
Size of Cache Block (Line):	32 bytes (words) = 2^5

Associative or Fully Associative Mapping

Consider Pentium Processor Cache

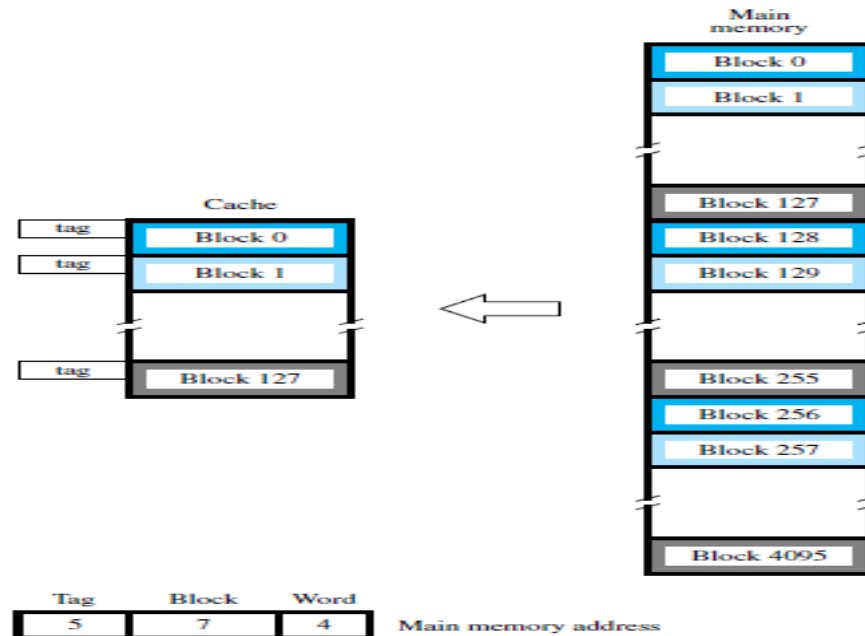
Size of Main Mem:	4GB = 2^{32}
Size of Cache Mem:	8KB = 2^{13}
Size of Cache Block (Line):	32 bytes (words) = 2^5

Tag: 27 bits

Word/offset=5bits

Direct Mapping or One-Way Set Associative Mapping

- 2048 or 2K words
- 16 words in a block



- 64K words
(4K blocks of 16 words each)

$(\text{Block address}) \text{ MOD } (\text{Number of blocks in cache})$

16 bit Address:

*00000**000000*10000

$$2^7 = 128$$
$$2^7 = 128$$

If match found, 4 least significant bits identify the word (out of $2^4 = 16$ possible words) in that particular line

Direct Mapping or One-Way Set Associative Mapping

Consider Pentium Processor Cache

Size of Main Mem:	4GB = 2^{32}
Size of Cache Mem:	8KB = 2^{13}
Size of Cache Block (Line):	32 bytes (words) = 2^5

Direct Mapping or One-Way Set Associative Mapping

Consider Pentium Processor Cache

Size of Main Mem:	4GB = 2^{32}
Size of Cache Mem:	8KB = 2^{13}
Size of Cache Block (Line):	32 bytes (words) = 2^5

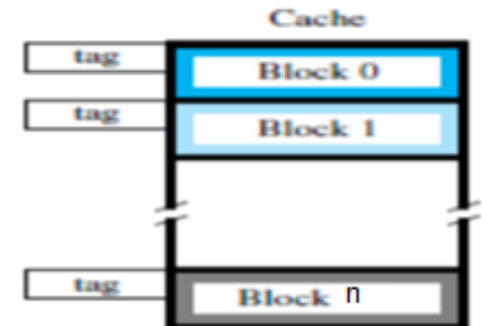
Tag: 19 bits

block/index=8 bits

Word/offset=5 bits

Problem

Assume a processor has a direct mapped cache with data words of 8 bits long. The number of addresses lines required to generate physical address for each word is 20 bits long. The tag of cache is 11 bits and each block holds 16 bytes of data. How many blocks are in this cache?



Problem

Assume a processor has a direct mapped cache with data words of 8 bits long. The number of addresses lines required to generate physical address for each word is 20 bits long. The tag of cache is 11 bits and each block holds 16 bytes of data. How many blocks are in this cache?

Solution:

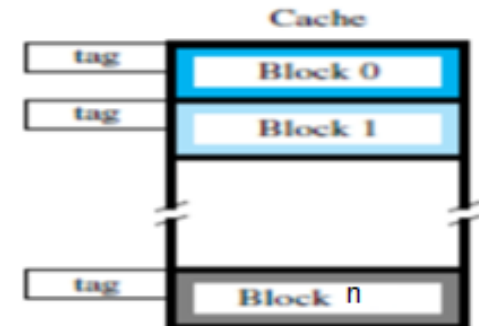
Given that the physical address is 20 bits long, and the tag is 11 bits, there are 9 bits left over for the index and offset

We can determine the number of bits of offset as the problem states that:

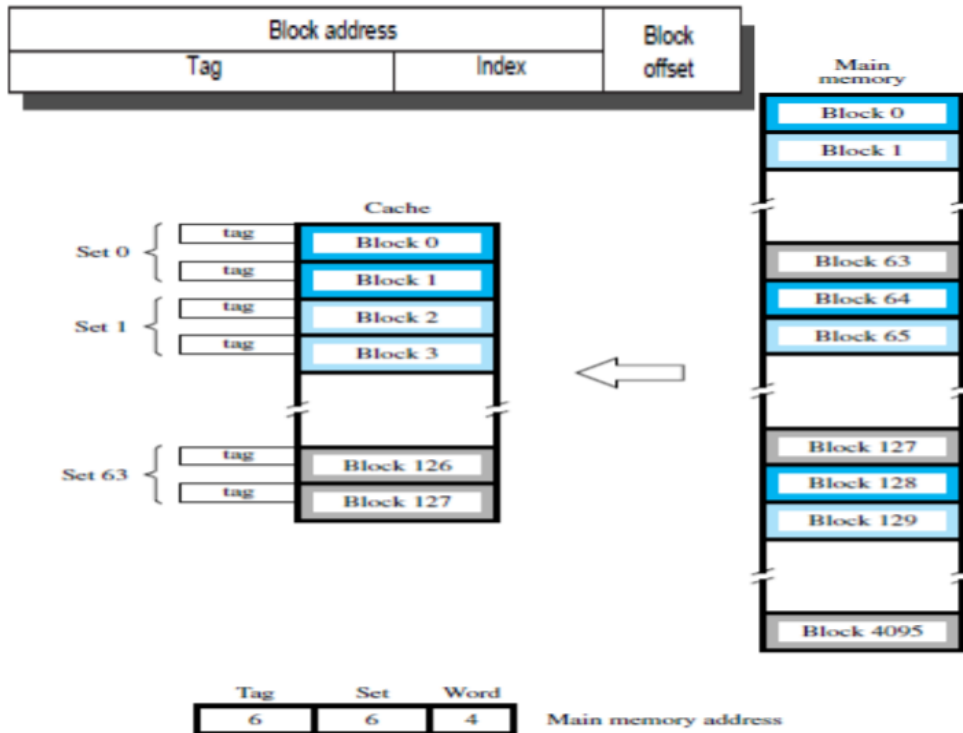
- Data is word addressable and words are 8 bits long
- Each block holds 16 bytes

As there are 8 bits / byte, each block holds 16 words, thus **4** bits of **offset** are needed.

This means that there are 5 bits left for the index. **Thus, there are 2^5 or 32 blocks in the cache.**



Set-Associative Mapping



If there are n blocks in a set, the cache placement is called *n -way set associative*.

NUMBER OF SETS IN THE CACHE = CACHE SIZE / SET SIZE = CACHE SIZE / SET ASSOCIATIVITY X BLOCK SIZE

No. of sets = $2k/16 \times 2 = 64$

No. of sets = No. of lines/blocks in cache / set associativity
 $= 128/2 = 64$

$(\text{Block address}) \text{ MOD } (\text{Number of sets in cache})$

Set-Associative Mapping or Two-Way Set Associative Mapping

Block address		Block offset
Tag	Index	

Consider Pentium Processor Cache

Size of Main Mem:	4GB = 2^{32}
Size of Cache Mem:	8KB = 2^{13}
Size of Cache Block (Line):	32 bytes (words) = 2^5

Set-Associative Mapping or Two-Way Set Associative Mapping

Block address		Block offset
Tag	Index	

Consider Pentium Processor Cache

Size of Main Mem:	4GB = 2^{32}
Size of Cache Mem:	8KB = 2^{13}
Size of Cache Block (Line):	32 bytes (words) = 2^5

Tag: 20 bits
Set/index=7 bits
Word/offset=5 bits

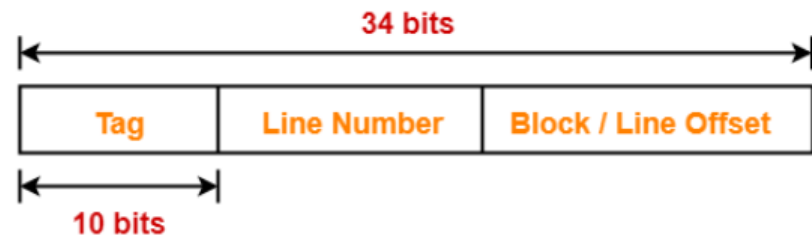
Example Problem 1 : Consider a direct mapped cache with block size 4 KB. The size of main memory is 16 GB and there are 10 bits in the tag. Find the Index and block offset fields . Also the Size of cache memory .

Example Problem 1 : Consider a direct mapped cache with block size 4 KB. The size of main memory is 16 GB and there are 10 bits in the tag. Find the Index and block offset fields . Also the Size of cache memory .

- Sol : Direct Mapped Cache
- Block Size = 4 KB
- Main memory Size = 16GB
- Block offset = $\log_2(\text{block size})$
- = 12 bits
- Physical Address = $\log_2(\text{Main memory Size})$
 - = 34 bits
 - Tag = 10 bits
 - Index = $34 - (12 + 10)$
 - = 12 bits
- Cache Size = block Size x no of blocks =

$$= 16\text{MB}$$

$$2^{12} \times 2^{12}$$



The address lines can be calculated as $2^m = n$

Ques: Consider a 2MB 4-way set-associative write back cache with 16 byte line (block) size and a 32 bit byte-addressable address. Assume a random replacement policy and a single core system. How many bits of the address are used for the cache index, tag and offset?

Ques: Consider a 2MB 4-way set-associative write back cache with 16 byte line (block) size and a 32 bit byte-addressable address. Assume a random replacement policy and a single core system. How many bits of the address are used for the cache index, tag and offset?

- Cache Size = 2MB
- Set Associativity = 4
- Physical Address = 32 bits
- Block size = 16 bytes
- Block offset = 4 bits
- Index = $\log_2(\text{no of sets in cache})$
- $= 2^{21} / 2^2 \times 2^4$
- = 15 bits
- Tag = $32 - (15 + 4)$
- = 13 bits

Tag (13bits)	Index(15bits)	Offset(4 bits)